

115 學年度資訊學科能力競賽臺南一中校內複選

115 學年度資訊學科能力競賽臺南一中校內複選 試 題本

競賽規則

1. 競賽時間：2026/09/24 13:00 ~ 17:30，共 4.5 小時。
2. 本次競賽試題共 7 題，每題皆有子任務。
3. 為了愛護地球，本次競賽題本僅提供電子檔，不提供紙本。
4. 每題的分數為該題所有子任務得分數加總；單筆子任務得分數為各筆繳交在該筆得到的最大分數。
5. 本次複選不提供記分板。
6. 除通訊題目以外其他題目的輸入皆為標準輸入。
7. 除通訊題目以外其他題目的輸出皆為標準輸出。
8. 所有輸入輸出請嚴格遵守題目要求，多或少的換行及空格皆有可能造成裁判系統判斷為答案錯誤。
9. 每題每次上傳間隔為 120 秒，裁判得視情況調整。
10. 所有試題相關問題請於競賽系統中提問，題目相關公告也會公告於競賽系統，請密切注意。
11. 如有電腦問題，請舉手向監考人員反映。
12. 如有如廁需求，須經過監考人員同意方可離場。
13. 不得攜帶任何參考資料，但競賽系統上的參考資料可自行閱讀。
14. 不得自行攜帶隨身碟，如需備份資料，請將資料儲存於電腦 D 槽。
15. 競賽中請勿交談。請勿做出任何會干擾競賽的行為。
16. 如需使用 C++ 的 `std::cin` 或 `std::cout` 可將以下程式碼插入 `main function` 以及將 `endl` 取代為 `'\n'` 來最佳化輸入輸出速度。唯須注意不可與 `cstdio` 混用。

```
std::ios::sync_with_stdio(false);  
std::cin.tie(nullptr);
```

A. 彩虹遊行

Problem ID: Rainbow_Parade

Time Limit: 1s

Memory Limit: 512MiB

家豪（Charhao）身為台南一中的三好學生，平時最喜歡翹課出來參加彩虹遊行。顧名思義，彩虹遊行指的是一大群彩虹小馬愛好者聚在一起的盛大活動。

今天的彩虹遊行在一座神奇的島嶼上舉辦，島上有 n 個彩虹小馬景點，編號為 $1 \sim n$ 。島嶼的入口位於 1 號景點，而開幕式場地則在 n 號景點。

家豪對這個島嶼一無所知，一個人走一定會迷路，所以他需要熱心助人的彩虹小馬學長 tw87 帶路。島上共有 m 條單向道路，第 i 條道路從景點 a_i 連向景點 b_i 。

由於 tw87 趕著去參加「彩虹小馬聖經研討會」，無法親自帶路，於是他決定畫一張地圖給家豪。地圖的繪製規則如下：

- 除了開幕式景點 n 之外，每個景點都必須**恰好選擇**一條從該點出發的單向道路，寫下下一個該去的景點。
- 為了避免地圖標示混亂，除了入口 1 號景點外，其餘每個景點在地圖上**恰好只能被一個景點指引**。
- 家豪必須能順利從入口 1 號景點沿著指引走到開幕式 n 號景點。但為了滿足 tw87 的惡趣味，對於**不在這條主線路徑上**的其他景點，tw87 刻意將它們設計成**若家豪不小心闖入，將會陷入無限繞圈圈的死循環中**。

每條道路 i 都有一個「奇怪值」 d_i 。只要 tw87 在地圖上記錄了該條道路，該道路的奇怪值就會累加到總和中。

tw87 想在滿足自己惡趣味的前提下，盡可能最小化總奇怪值，以免被可愛的學弟討厭。身為一中的彩虹新秀，請你幫 tw87 計算出最小可能的總奇怪值為多少？

— 輸入 —

第一行輸入二正整數 n m

接下來 m 行，輸入二正整數與一非負整數 a_i, b_i, d_i

— 輸出 —

輸出一個整數，代表最小可能的總奇怪值。

若無法構造出滿足條件的地圖，則輸出 -1。

— 輸入限制 —

- $2 \leq n \leq 2000$
- $1 \leq m \leq \min(10^4, n \times (n - 1))$
- $1 \leq a_i, b_i \leq n, a_i \neq b_i$
- $0 \leq d_i \leq 10^5$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	8	$2 \leq n \leq 8, 1 \leq m \leq \min(30, n \times (n - 1)), 0 \leq d_i \leq 100$
3	17	$2 \leq n \leq 18, 1 \leq m \leq \min(200, n \times (n - 1)), 0 \leq d_i \leq 100$
4	22	$2 \leq n \leq 1000, 1 \leq m \leq \min(5000, n \times (n - 1)), 0 \leq d_i \leq 1$
5	24	$2 \leq n \leq 200, 1 \leq m \leq \min(1000, n \times (n - 1)), 0 \leq d_i \leq 10^5$
6	29	無額外限制

— 範例一輸入 —

4 6
1 2 5
1 3 1
2 3 2
2 4 2
3 2 3
3 4 4

— 範例一輸出 —

6

— 範例一說明 —

若選擇道路 $1 \rightarrow 3$ 、 $3 \rightarrow 2$ 、 $2 \rightarrow 4$ ，家豪會沿著 $1 \rightarrow 3 \rightarrow 2 \rightarrow 4$ 抵達開幕式，總奇怪值為 $1 + 3 + 2 = 6$ 。

另一種可行選法是 $1 \rightarrow 2$ 、 $2 \rightarrow 3$ 、 $3 \rightarrow 4$ ，總奇怪值為 $5 + 2 + 4 = 11$ ，因此最小可能總和為 6。

— 範例二輸入 —

3 2
1 2 4
1 3 1

— 範例二輸出 —

-1

— 範例二說明 —

景點 2 不是開幕式景點，卻沒有任何從景點 2 出發的道路，無法替它選擇指引。因此無法構造符合條件的地圖，答案為 -1。

— 範例三輸入 —

5 8
1 4 2
4 5 2
2 3 1
3 2 1
1 2 10
2 4 10
3 5 10
4 3 10

— 範例三輸出 —

6

— 範例三說明 —

可以選擇道路 $1 \rightarrow 4$ 、 $4 \rightarrow 5$ 作為主線，並讓其餘景點形成 $2 \rightarrow 3 \rightarrow 2$ 的循環，總奇怪值為 $2 + 2 + 1 + 1 = 6$ 。

任何合法地圖都必須選擇 $n - 1 = 4$ 條道路，而本例中四條最小的道路奇怪值總和已是 6，所以不可能得到更小的答案。

B. 小雞過馬路

Problem ID: Chicks_Go

Time Limit: 1s

Memory Limit: 512MiB

車輛來來去去的馬路上有一條斑馬線。

這條斑馬線的每個白色區塊（共 n 個）可以視為在一維數線上依序排列，位置依序處於 $x_1, x_2, \dots, x_n$ 。

今天有 m 隻小雞位於第一塊白色區塊（ x_1 ），想要到達最後一塊區塊（ x_n ）後，再折返回第一塊區塊。

然而，斑馬線上的白色區塊會因為小雞們的踩踏而磨損。除了第一塊與最後一塊區塊異常堅固（不會磨損）之外，對於中間的任何一個白色區塊 i ，只要有一隻小雞跳到上面，它的被踩踏次數就會加 1。當該區塊總共被踩踏了 a_i 次後，就會變成與路面一樣的顏色而消失（不再屬於斑馬線）。

小雞們是很遵守交通規則的，所以牠們無論去程或回程，都只會踩在還沒消失的白色區塊上。假設每隻小雞的跳躍能力相同，請你幫忙計算：為了讓 m 隻小雞都能順利完成往返目標，小雞們的「單次最大跳躍距離」至少需要是多少？

— 輸入 —

第一行輸入二正整數 n m

第二行輸入 n 個整數 $x_1, x_2, \dots, x_n$

第三行輸入 $n - 2$ 個正整數 $a_2, a_3, \dots, a_{n-1}$

— 輸出 —

輸出小雞們的「單次最大跳躍距離」至少需要是多少。

— 輸入限制 —

- $3 \leq n \leq 10^5$
- $1 \leq m \leq 10^5$
- $|x_i| \leq 10^9$
- $x_1 < x_2 < \dots < x_n$
- $1 \leq a_i \leq 10^9$

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	7	$m = 1, a_i = 10^9$
3	18	$3 \leq n \leq 2000, m = 1, a_i = 1$
4	9	$3 \leq n \leq 7, 1 \leq m \leq 7$
5	20	$3 \leq n \leq 100, 1 \leq m \leq 100, \sum a_i \leq 10^4$
6	22	$3 \leq n \leq 2000$
7	24	無額外限制

— 範例一輸入 —

3 1
0 1 2
1

— 範例一輸出 —

2

— 範例一說明 —

若最大跳躍距離小於 2，小雞去程時必須踩到中間的白色區塊，使它在被踩踏 1 次後消失，回程便無法再使用它。

當最大跳躍距離為 2 時，小雞可以在去程與回程都直接往返第一塊與最後一塊白色區塊，因此答案為 2。

— 範例二輸入 —

```
5 2
-5 -1 2 3 10
3 1 5
```

— 範例二輸出 —

```
7
```

— 範例二說明 —

把兩隻小雞的去程與回程視為四次從 x_1 到 x_n 的移動（回程路線反向使用）。其中三次可以依序踩過第 1, 2, 4, 5 塊白色區塊，另一次依序踩過第 1, 3, 4, 5 塊。所有跳躍距離都不超過 7，且各中間區塊的踩踏次數不超過限制。

最後兩塊白色區塊的距離為 $10 - 3 = 7$ ，每隻小雞都一定要跨過這段距離，所以答案不可能小於 7。

— 範例三輸入 —

```
6 3
-20 -19 -4 0 1 30
6 1 5 1000000000
```

— 範例三輸出 —

```
29
```

— 範例三說明 —

三隻小雞共有六次單程移動。每次都可以依序踩過第 1, 2, 5, 6 塊白色區塊；第 2 塊恰好被踩踏 6 次，而第 5 塊的耐久度也足夠。這些路線的最大跳躍距離為 $30 - 1 = 29$ 。

因為最後兩塊白色區塊相距 29，所有路線都必須跨過這段距離，所以答案為 29。

C. 一棵簡單的樹

Problem ID: Simple_Tree

Time Limit: 1s

Memory Limit: 512MiB

給定一棵以節點 1 為根、包含 N 個節點的樹。節點 i 上一開始有 A_i 顆石頭。

Alice 與 Bob 輪流行動，由 Alice 先手。每次行動可以選擇一個不是根、且仍有石頭的節點，從中拿走任意正整數顆石頭，並將這些石頭全部移到該節點的親代。無法行動的人輸掉遊戲。

— 輸入 —

第一行包含一個整數 N ，代表樹的節點數。

第二行包含 N 個整數 $A_1, A_2, \dots, A_N$ ，其中 A_i 代表節點 i 一開始的石頭數量。

接下來 $N - 1$ 行，每行包含兩個整數 u_i, v_i ，代表節點 u_i 與節點 v_i 之間有一條無向邊。將樹以節點 1 為根後，每個非根節點的親代即為它通往節點 1 的路徑上的下一個節點。

— 輸出 —

若 Alice 有必勝策略，輸出 Alice；否則輸出 Bob。

— 輸入限制 —

- $1 \leq N \leq 2 \times 10^5$
- $0 \leq A_i \leq 10^9$
- $1 \leq u_i, v_i \leq N$
- 輸入的 $N - 1$ 條邊構成一棵樹。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	10	$N \leq 2$
3	15	除了根節點外，每個節點的親代皆為節點 1
4	20	對所有 $1 \leq i < N$ ，第 i 條邊連接節點 i 與 $i + 1$
5	20	對所有 $1 \leq i \leq N$ ， $0 \leq A_i \leq 1$
6	35	無額外限制

— 範例輸入 1 —

```
2
1 1
1 2
```

— 範例輸出 1 —

```
Alice
```

— 範例說明 1 —

Alice 可以把節點 2 上唯一的一顆石頭移到根節點。此後所有非根節點都沒有石頭，Bob 無法行動，因此 Alice 獲勝。

－ 範例輸入 2 －

```
3
0 5 5
1 2
1 3
```

－ 範例輸出 2 －

Bob

－ 範例說明 2 －

節點 2 與節點 3 的位置對稱，且一開始都各有 5 顆石頭。每當 Alice 從其中一個節點移動若干顆石頭時，Bob 都可以從另一個節點移動相同數量的石頭。如此能維持兩個節點的石頭數量相同，最後 Alice 會先無法行動，因此 Bob 獲勝。

－ 範例輸入 3 －

```
5
100 6 0 0 10
1 2
2 3
2 4
1 5
```

－ 範例輸出 3 －

Alice

－ 範例說明 3 －

Alice 先將節點 5 上的 4 顆石頭移到根節點，使節點 2 與節點 5 都剩下 6 顆石頭。此時其他非根節點都沒有石頭。

之後每當 Bob 從節點 2 或節點 5 移動若干顆石頭時，Alice 都能從另一個節點移動相同數量的石頭。最後 Bob 會先無法行動，因此 Alice 獲勝。

D. Merge, Rebase, Squash!

Problem ID: Git

Time Limit: 2.5s

Memory Limit: 512MiB



Force push 1



Force push 2

Figure 1: 不要學 DB force push (他現在已經不會了)

tobiichi3227 是 TOJ 的維護者，TOJ 整個程式碼由 H 個獨立的 **hunk** 組成，每個 hunk 的狀態一開始都是 0，在 v2.0 分支上面。

有天王德宏想給 TOJ 貢獻一個酷炫的功能，於是王德宏很高興的開啟了一個新的分支 feat/fancy 並在上面建立了 N 個 commit。

王德宏想讓功能進到 TOJ 的 v2.0，於是請 tobiichi3227 來 review，tobiichi3227 覺得 N 個 commit 太多了，應該用 squash 把 commit 減少到只有 K 個。

當 squash 完成後，會將 feat/fancy rebase 到 v2.0，王德宏發現會有 conflict，所以他想要最小化解決 conflict 的成本，但王德宏要去比國際地理奧林匹亞 (IGGO, 全名 International GeoGuesser Olympiad) 沒空了，於是交給在看題目的你。



Figure 2: Branch、squash、rebase 與 conflict resolution 的執行過程

若使用支援 PDF 附件的 Firefox，可
分鏡。

；Chrome 與 Edge 請直接閱讀上方

— Branch 狀態 —

v2.0 branch 上第 h 個 hunk 的最終內容為 U_h 。

feat/fancy branch 上共有 N 個 commits。第 i 個 commit 會將第 x_i 個 hunk 的內容改成 v_i 。

令 $S_i[h]$ 表示執行完 feat/fancy branch 的前 i 個 commits 後，第 h 個 hunk 的內容。

初始時： $S_0[h] = 0$

第 i 個 commit 執行後： $S_i[x_i] = v_i$

其他 hunk 的內容不變。

– Squash –

原本的 N 個 commits 依照順序排列：

commit 1, commit 2, ..., commit N

你需要在這些 commits 之間放置 $K - 1$ 個切點，將它們分成恰好 K 組。

每一組中的所有 commits 會被合併成一個新的 **squash commit**。

例如，假設共有 7 個 commits，且要分成 3 組，可以選擇：

```
[commit 1, commit 2]
[commit 3, commit 4, commit 5]
[commit 6, commit 7]
```

也可以簡寫成：

```
[1, 2] [3, 4, 5] [6, 7]
```

這代表：

- 第一個 squash commit 包含原本的 commits 1 到 2
- 第二個 squash commit 包含原本的 commits 3 到 5
- 第三個 squash commit 包含原本的 commits 6 到 7

為了描述一般情況，令： $0 = p_0 < p_1 < p_2 < \dots < p_K = N$

其中 p_g 表示第 g 組的最後一個 commit 編號。

因此，第 g 個 squash commit 包含： $p_{g-1} + 1, p_{g-1} + 2, \dots, p_g$

例如前面的分組：

```
[1, 2] [3, 4, 5] [6, 7]
```

對應： $p_0 = 0, p_1 = 2, p_2 = 5, p_3 = 7$

考慮第 g 個 squash commit，以及某個 hunk h 。

在這組 commits 開始以前，hunk h 的內容是： $S_{p_{g-1}}[h]$

執行完這組中的所有 commits 後，hunk h 的內容是： $S_{p_g}[h]$

因此，這個 squash commit 對 hunk h 的效果為： $S_{p_{g-1}}[h] \rightarrow S_{p_g}[h]$

也就是只保留這一組 commits 執行前與執行後的差異，中間經過的狀態不會出現在 squash commit 中。

例如，某個 hunk 在一組 commits 中依序發生：

1 -> 3 -> 5 -> 2

那麼 squash 後只會留下：

1 -> 2

中間的 3 與 5 都不會出現在 squash commit 中。

若一組 commits 執行前後，某個 hunk 的內容相同，即： $S_{p_{g-1}}[h] = S_{p_g}[h]$

則這個 squash commit 不會修改該 hunk。

– Rebase 與 Conflict –

完成 squash 後，tobiichi3227 會依照原本的順序，將這 K 個 squash commits 逐一套用到 v2.0 branch 上。

考慮其中一個 squash commit 對 hunk h 的修改： $\text{old} \rightarrow \text{new}$

其中：
- old：這個 squash commit 預期修改前的內容；
- new：這個 squash commit 想要修改成的內容；
- current：目前 v2.0 branch 上實際存在的內容。

例如： $\text{old} = 3$, $\text{new} = 7$, $\text{current} = 5$

代表這個 squash commit 原本是在內容為 3 的基礎上，將它改成 7，但目前 v2.0 branch 上的內容實際是 5。

沒有修改

若： $\text{old} = \text{new}$

代表這個 squash commit 執行前後，該 hunk 的內容沒有改變。

因此不需要進行任何操作，current 也不會改變。

情況一：目前內容正是預期的舊內容

若： $\text{current} = \text{old}$

代表目前內容與這個 patch 預期的起點相同，因此可以正常套用修改。

套用後： $\text{current} \leftarrow \text{new}$

例如：

```
patch:    3 -> 7
current: 3
```

可以正常套用，完成後 current 變成 7。

情況二：目前內容已經是目標內容

若： $\text{current} = \text{new}$

代表 v2.0 branch 上已經存在這個 squash commit 想要產生的結果。

因此不會發生 conflict，也不需要再次修改。

例如：

```
patch:    3 -> 7
current: 7
```

雖然目前內容不是 patch 預期的舊內容 3，但它已經等於目標內容 7，所以可以直接視為這項修改已經存在。

current 維持為 7。

情況三：目前內容既不是舊內容，也不是新內容

若： $\text{current} \neq \text{old}$

且： $\text{current} \neq \text{new}$

代表 v2.0 branch 上的內容與這個 squash commit 預期的內容不同，而且也不是它想要修改成的結果。

此時會發生 conflict，例如：

patch: 3 -> 7

current: 5

目前內容 5 既不是舊內容 3，也不是新內容 7，因此發生 conflict。

解決 hunk h 的 conflict 需要支付 w_h 的成本。

解決 conflict 後，採用 squash commit 的修改結果： $\text{current} \leftarrow \text{new}$

在前面的例子中，解決後 current 會變成 7。

同一個 squash commit 可能同時修改多個 hunks。每個 hunk 是否發生 conflict 都要分別判斷，發生 conflict 的成本也要分別計算。

— 輸入 —

第一行包含三個整數 N, H, K ，分別表示：

- feat/fancy branch 上的 commit 數量；
- hunk 的數量；
- squash 後必須保留的 commit 數量。

第二行包含 H 個整數 U_h ($1 \leq h \leq H$)

其中 U_h 表示 v2.0 branch 上第 h 個 hunk 的最終內容。

第三行包含 H 個整數 w_h ($1 \leq h \leq H$)

其中 w_h 表示解決第 h 個 hunk 的 conflict 所需的成本。

接下來 N 行，第 i ($1 \leq i \leq N$) 行包含兩個整數 x_i 與 v_i
表示第 i 個 commit 將第 x_i 個 hunk 的內容改成 v_i 。

— 輸出 —

輸出一個整數，表示最小的 conflict 解決總成本。

— 輸入限制 —

- $1 \leq N, H \leq 200000$
- $1 \leq K \leq \min(N, 30)$
- $1 \leq U_h \leq 10^9$
- $1 \leq w_h \leq 10^9$
- $1 \leq x_i \leq H$
- $1 \leq v_i \leq 10^9$
- 對於每個 commit $i : v_i \neq S_{i-1}[x_i]$
- 保證每個 commit 都會實際改變該 hunk 的內容，且 feat/fancy branch 不會將任何 hunk 改回 0。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	8	$N, H \leq 15$
3	8	$K = 1$
4	8	$K = N$
5	10	每個 hunk 最多只被修改一次
6	26	$N \leq 1000$
7	40	無限制

— 範例輸入 —

```
5 2 2
1 4
5 3
1 1
2 2
1 2
2 4
1 3
```

— 範例輸出 —

```
0
```

— 範例說明 —

feat/fancy branch 上兩個 hunk 的狀態依序如下：

```
commit 0: (0, 0)
commit 1: (1, 0)
commit 2: (1, 2)
commit 3: (2, 2)
commit 4: (2, 4)
commit 5: (3, 4)
```

可以將 commits 分成：

[1] [2, 3, 4, 5]

第一個 squash commit 將 hunk 1 :

0 -> 1

套用到 v2.0 branch 時，hunk 1 的目前內容已經是 1，因此不會發生 conflict。

第二個 squash commit 的效果為：

hunk 1 : 1 -> 3

hunk 2 : 0 -> 4

此時 hunk 1 的目前內容為 1，可以正常套用。

hunk 2 的目前內容已經是 4，因此也不會發生 conflict。

總成本為 0。

— 範例輸入 —

```
2 1 1
1
7
1 1
1 2
```

— 範例輸出 —

```
7
```

— 範例說明 —

因為 $K = 1$ ，兩個 commits 必須全部 squash 在一起。
Squash 後的效果為：

```
0 -> 2
```

但 v2.0 branch 上該 hunk 的內容為 1。

目前內容既不是 patch 的舊內容 0，也不是新內容 2，因此發生 conflict，成本為 7。

— 範例輸入 —

```
2 1 2
1
7
1 1
1 2
```

— 範例輸出 —

```
0
```

— 範例說明 —

因為 $K = 2$ ，兩個 commits 分別套用。
第一個 commit 的效果為：

```
0 -> 1
```

而 v2.0 branch 上目前的內容已經是 1，因此不會發生 conflict。

第二個 commit 的效果為：

1 -> 2

此時目前內容為 1，可以正常套用。

因此總成本為 0。

— Git 小教室 —

下面會用一個小型 repository 實際重現題目中的 branch、squash、rebase 與 conflict。整份教學都能在**沒有網路**的環境執行：toj.git 是建立在同一台電腦上的本機 remote，不會連線到 GitHub。命令以 Bash 或 Git Bash 為例；程式碼框中的命令不用輸入開頭的提示符號。

Git 在記錄什麼？

- **working tree**：目前資料夾中實際看到的檔案。
- **staging area**：下一個 commit 準備收錄的內容。
- **commit**：repository 在某個時間點的快照，以及指向前一個 commit 的資訊。
- **branch**：指向某個 commit、並會隨新 commit 前進的名稱，例如 v2.0、feat/fancy。
- **HEAD**：目前 checkout 的 branch 或 commit。
- **remote**：另一個 repository 的別名；remote 可以在網路上，也可以只是本機路徑。

最常使用的命令如下：

git init	建立 repository
git status	查看目前 branch、已修改與已 stage 的檔案
git diff	查看尚未 stage 的差異
git diff --staged	查看已 stage、將進入下一個 commit 的差異
git add <file>	將檔案的目前內容放進 staging area
git commit -m "<message>"	建立 commit
git log --graph --oneline --all	用簡圖查看所有 branch 的歷史
git branch	列出 branch
git switch <branch>	切換 branch
git switch -c <branch>	建立並切換到新 branch
git merge <branch>	把指定 branch 合併到目前 branch
git rebase <branch>	把目前 branch 的 commits 逐一重播到指定 branch

`git commit -am "<message>"` 是 `git add` 已追蹤檔案再 `git commit` 的簡寫，但**不會加入新建立、尚未追蹤的檔案**。不確定目前狀態時，先執行 `git status` 通常最安全。

1. 建立完全離線的 repository

先建立一個新的空資料夾。`toj.git` 模擬遠端的 TOJ repository，`work` 則是實際工作的 repository：

```
mkdir git-classroom
cd git-classroom
git init --bare toj.git
git init -b v2.0 work
cd work
```

bare repository 沒有 working tree，適合當作大家交換 commits 的 remote。接著只替這個練習設定作者資料；`example.invalid` 是保留給範例使用的網域，不需要網路：

```
git config user.name "Git Student"
git config user.email "student@example.invalid"
```

用兩個一行文字檔模擬兩個 hunk，並建立共同的起始 commit：

```
printf "0\n" > hunk-1.txt
printf "0\n" > hunk-2.txt
git add hunk-1.txt hunk-2.txt
git status
git commit -m "base: all hunks are 0"
git tag base
```

base tag 只是替這個 commit 取一個固定名稱，之後進行 squash 時便不必抄寫隨機產生的 commit hash。

2. 在 feat/fancy 建立四個 commits

從共同起點建立新 branch：

```
git switch -c feat/fancy
```

依序建立四個 commits。這對應題目中同一個 hunk 可能被多次修改：

```
printf "3\n" > hunk-1.txt
git commit -am "fancy 1: hunk 1, 0 to 3"
```

```
printf "4\n" > hunk-2.txt
git commit -am "fancy 2: hunk 2, 0 to 4"
```

```
printf "5\n" > hunk-1.txt
git commit -am "fancy 3: hunk 1, 3 to 5"
```

```
printf "7\n" > hunk-1.txt
git commit -am "fancy 4: hunk 1, 5 to 7"
```

可以查看目前的四個 feature commits：

```
git log --oneline --decorate
```

將本機 bare repository 命名為 origin，並保存 squash 前的 branch：

```
git remote add origin ../toj.git
git push -u origin feat/fancy
```

-u 會記住本機 feat/fancy 對應到 origin/feat/fancy。這裡的 remote 是 ../toj.git，因此 push 全程只會存取本機檔案。

3. 讓 v2.0 同時向前進

切回 v2.0，令它的兩個 hunk 最終為 (5, 4)：

```
git switch v2.0
printf "5\n" > hunk-1.txt
printf "4\n" > hunk-2.txt
git commit -am "v2.0: set final hunk states"
git push -u origin v2.0
git log --graph --oneline --decorate --all
```

現在兩個 branches 從 base 分岔：

```

        fancy 1 -- fancy 2 -- fancy 3 -- fancy 4 (feat/fancy)
      /
base: (0, 0) --
      \
        v2.0: (5, 4) (v2.0)

```

4. Merge 與 Rebase 有什麼不同？

merge 會把另一條 branch 的最終結果合進目前 branch；若兩邊都有新 commits，通常會另外建立一個有兩個 parents 的 merge commit。先建立不影響正式流程的 merge-demo：

```
git switch -c merge-demo v2.0
git merge feat/fancy
```

兩邊都把 hunk-1.txt 從 0 改成不同內容，因此 Git 會回報 conflict。兩邊都把 hunk-2.txt 改成 4，所以該檔案可以自動合併。用以下命令可以查看未解決的檔案：

```
git status
git diff
```

這次只是在比較 merge，所以放棄它並回到 feature branch：

```
git merge --abort
git switch feat/fancy
```

相對地，rebase v2.0 會找出 feat/fancy 與 v2.0 的共同祖先，將 feature commits 暫時取下，再按原順序逐一套用到新版 v2.0 後面。因此 rebase 後的歷史通常較直線；在這個已經分岔、確實需要 replay 的例子中，commit 的 parent 與內容雜湊改變，commit hash 也會改變。

5. 把四個 commits squash 成兩個

題目要求把連續的 commits 分組。這裡示範 [1, 2] [3, 4]，也就是令 $K = 2$ 。在 feat/fancy 上執行互動式 rebase：

```
git rebase -i base
```

編輯器會依照由舊到新的順序列出四行，commit hash 會因電腦與執行時間而不同：

```
pick <hash 1> fancy 1: hunk 1, 0 to 3
pick <hash 2> fancy 2: hunk 2, 0 to 4
pick <hash 3> fancy 3: hunk 1, 3 to 5
pick <hash 4> fancy 4: hunk 1, 5 to 7
```

把每一組中除了第一個以外的 pick 改成 squash：

```
pick    <hash 1> fancy 1: hunk 1, 0 to 3
squash  <hash 2> fancy 2: hunk 2, 0 to 4
pick    <hash 3> fancy 3: hunk 1, 3 to 5
squash  <hash 4> fancy 4: hunk 1, 5 to 7
```

儲存並關閉編輯器；若 Git 再要求編輯合併後的 commit message，確認內容後同樣儲存並關閉。第一個 squash commit 的效果為：

```
hunk 1: 0 -> 3
hunk 2: 0 -> 4
```

第二個 squash commit 的效果為：

```
hunk 1: 3 -> 7
```

中間的 3 -> 5 -> 7 已經縮成 3 -> 7。確認 base 後面現在恰有兩個 commits：

```
git rev-list --count base..feat/fancy
git log --oneline --reverse base..feat/fancy
```

第一個命令應輸出 2。真實 repository 若沒有事先建立 base tag，可以先用 git merge-base v2.0 feat/fancy 找共同祖先，再將輸出的 commit hash 交給 git rebase -i。

6. Rebase 並解決 conflict

把這兩個 squash commits 逐一 replay 到 v2.0：

```
git rebase v2.0
```

套用第一個 squash commit 時：

- hunk-1 的 patch 是 0 -> 3，但 v2.0 的 current 是 5，因此發生 conflict。
- hunk-2 的 patch 是 0 -> 4，而 v2.0 的 current 已經是 4，因此不發生 conflict。

Git 會停下來等待處理。查看狀態與 conflict markers：

```
git status
cat hunk-1.txt
cat hunk-2.txt
```

hunk-1.txt 會包含以下幾個部分（左欄的說明文字不在實際檔案中）：

```
目前版本的起點標記 | <<<<<<< HEAD
目前版本的內容      | 5
分隔線              | =====
正在 replay 的內容   | 3
feature 版本的標記   | >>>>>>> fancy 1: hunk 1, 0 to 3
```

<<<<<<<、=====、>>>>>>> 是 Git 加入的標記，不是檔案原本的內容。題目規定解決 conflict 後採用 squash commit 的 new，所以刪除標記並把檔案內容改成 3：

```
printf "3\n" > hunk-1.txt
git add hunk-1.txt
git rebase --continue
```

git add 在這裡代表「這個檔案的 conflict 已處理完成」。若 --continue 開啟 commit message 編輯器，直接確認、儲存並關閉即可。接著第二個 squash commit 的 3 -> 7 可以正常套用，rebase 便會完成。

```
git status
cat hunk-1.txt
cat hunk-2.txt
git rev-list --count v2.0..feat/fancy
git log --graph --oneline --decorate v2.0 feat/fancy
```

最後應得到 `hunk-1.txt = 7`、`hunk-2.txt = 4`，而且 `v2.0` 後面恰有兩個 `feature commits`。若想放棄整次 `rebase` 並回到開始前，可以在尚未完成時使用：

```
git rebase --abort
```

不要在不了解目前 `commit` 的情況下使用 `git rebase --skip`；它會直接丟掉正在 `replay` 的 `commit`。

7. 為什麼 `rebase` 後需要 `force push`？

`remote` 上仍保存 `squash` 前的四個 `commits`，但本機已把它們改寫成另外兩個 `commits`。普通 `push` 只允許 `fast-forward`，因此以下命令應被拒絕：

```
git push origin feat/fancy
```

確認沒有人在 `remote` 上加入你尚未看過的 `commit` 後，使用：

```
git push --force-with-lease origin feat/fancy
```

`--force-with-lease` 只會在 `remote branch` 仍是本機所預期的版本時覆寫它；若別人已經更新 `remote`，`push` 會被拒絕。它通常比無條件覆寫的 `--force` 安全，但改寫共享 `branch` 前仍應先和其他協作者確認。

題目模型與真實 `Git` 的差異

本題刻意把程式碼簡化成互相獨立的 `hunks`，並用 `old`、`new`、`current` 三個整數定義是否 `conflict`。真實 `Git` 會考慮檔案中的多行 `context`、相鄰修改、檔案新增刪除與重新命名等資訊，不能在所有情況下只靠三個值完整描述。上面的練習把每個 `hunk` 放在獨立的一行文字檔中，因此能重現本題所描述的三種結果；解題時仍應以題目正式定義為準。

E. 你可以幫我吹口哨嗎?

Problem ID: Can_You_Blow_My_Whistle

Time Limit: 1s

Memory Limit: 512MiB



Figure 1: Can You Blow My Whistle Baby

本題採用函式互動形式。

國中營比賽當天，中午吃飯的時候，有 n 個小學弟排成一列準備領便當。

小學弟們身上穿著 1 到 n 號的衣服，且每個號碼恰好出現一次。

目前他們站在位置 1 到 n 上，位於位置 i 的學弟是 a_i 。

身為大學長的你，希望他們能依照號碼 1 到 n 的順序從小到大排好（也就是位置 i 要站的是編號 i 的學弟）。

為了節省時間，你決定吹哨子發號施令。每次吹哨，你可以同時指定任意多組學弟互換位置，但為了避免碰撞，每個學弟在同一次吹哨中，最多只能參與一組交換。

身為口哨先輩的你，請安排交換方式，用盡可能少的哨音讓所有學弟排好。

— 函式介面 —

你只需要提交實作 `solve` 函式的程式碼，不可以實作 `main` 函式。評測系統會將你的程式與 `grader` 一起編譯。

你的程式應包含以下標頭檔：

```
#include "Can_You_Blow_My_Whistle.h"
```

評測系統會呼叫：

- `void solve(int n, std::vector<int> a);`
 - a 的長度為 n ，且是 1 到 n 的一個排列。
 - 對於 $0 \leq i < n$ ， $a[i]$ 表示一開始站在位置 $i + 1$ 的學弟編號。
 - 你需要在此函式中呼叫下列函式安排交換；不需要回傳任何值。

你可以呼叫：

- `void swap_student(int u, int v);`
 - $1 \leq u, v \leq n$ 且 $u \neq v$ 。
 - 將「交換位置 u 與位置 v 」加入目前這一輪，但此時還不會立刻交換。
 - 同一輪中，每個位置最多只能出現在一次 `swap_student` 呼叫中，否則會得到 **Wrong Answer(1)**。
- `void blow_whistle();`
 - 同時執行目前這一輪加入的所有交換，接著開始新的一輪。
 - 即使這一輪沒有加入任何交換，呼叫此函式仍會計入一次哨音。

傳入的 a 是普通的 `std::vector<int>`；呼叫上述函式不會自動修改你程式中的 a 。
若 `solve` 結束時仍有尚未經過 `blow_whistle` 執行的交換，這些交換不會生效。

— 輸入限制 —

- $1 \leq n \leq 10^5$
- $1 \leq a_i \leq n$
- a 是一個排列

— 給分方式 —

假如你呼叫 `blow_whistle` 函式的次數超過了 $2n$ 次，則系統會中斷，並給出 **Wrong Answer(2)**。

當你的 `solve` 函式結束時，系統會檢查學弟們是否已經正確排序。若最終沒有完成排序，該筆測資的得分倍率為 0。

若成功完成排序，令該筆測資理論上最少需要的哨音數為 opt ，你呼叫 `blow_whistle` 的總次數為 A ，該筆測資的得分倍率 T 為：

- 若 $A = opt$ ，則 $T = 1$ 。
- 若 $A > opt$ ，則 $T = \max(0, 0.5 - 0.02(A - opt))$ 。

也就是說，只要哨音數不是最佳解，該筆測資的倍率最多為 0.5，且每多吹一次會再扣除 0.02，直到降為 0。

對於滿分為 P 的子任務，令 $T_{\min}$ 為你在該子任務所有測資中得到的最小倍率，則此子任務的得分為 $P \times T_{\min}$ 。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	34	$1 \leq n \leq 2000$
3	15	保證 $opt = 1$
4	51	無額外限制

— 範例 —

評測系統會以 `solve(5, {2, 1, 3, 5, 4})` 呼叫你的函式。

例如可以依序呼叫：

```
swap_student(1, 2);  
swap_student(4, 5);  
blow_whistle();
```

前兩個呼叫將兩組交換加入同一輪；呼叫 `blow_whistle` 後，兩組交換同時執行，排列變成 `[1, 2, 3, 4, 5]`。此例最少需要一次哨音。

F. TODO: 題目名稱

Problem ID: TODO:_English_short_name

Time Limit: 1s

Memory Limit: 512MiB

TODO: 題目敘述

— 輸入 —

第一行包含兩個整數 N, Q ，分別代表點的數量與操作數量。

第二行包含 N 個整數 $A_1, A_2, \dots, A_N$ 。所有點的 B_i 初始皆為 0，因此不會在輸入中給定。

接下來 Q 行，每行為下列其中一種格式：

- 1 a b：若 a, b 位於不同連通塊，合併這兩個連通塊。
- 2 a b：若 a, b 位於不同連通塊，將點 a 從原連通塊移除，並加入 b 所在的連通塊。若原連通塊因此沒有任何點，該連通塊消失。
- 3 l r x：對所有滿足 $l \leq A_i \leq r$ 的點，令 B_i 增加 x 。
- 4 a：詢問點 a 所在連通塊中所有點的 B_i 總和。
- 5：撤銷最近一次成功執行、且尚未被撤銷的操作 1, 2, 3。

操作 1, 2 只有在 a, b 原本位於不同連通塊時才算成功。操作 3 只有在 $x \neq 0$ ，且至少存在一個點滿足 $l \leq A_i \leq r$ 時才算成功。只有成功的操作 1, 2, 3 會進入回溯紀錄；操作 4 不會進入回溯紀錄。

— 輸出 —

對每個第 4 種操作，依序輸出一行，包含一個整數，代表該連通塊中所有點的 B_i 總和。

— 輸入限制 —

- $1 \leq N, Q \leq 5 \times 10^4$
- $0 \leq A_i \leq 10^9$
- $1 \leq a, b \leq N$
- $0 \leq l \leq r \leq 10^9$
- $-10^9 \leq x \leq 10^9$
- 每次執行操作 5 時，至少存在一個尚未被撤銷的成功操作。
- 所有第 4 種操作的答案皆位於 signed 64-bit integer 可表示的範圍內。

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	15	$N, Q \leq 2000$

編號	分數	額外限制
3	10	只出現第 3, 4, 5 類操作
4	10	$A_1 = A_2 = \cdots = A_N$
5	15	A_i 中不同的值不超過 32 種
6	20	不出現第 2 類操作
7	30	無額外限制

— 範例一輸入 —

```
1 5
5
4 1
3 5 5 -7
4 1
5
4 1
```

— 範例一輸出 —

```
0
-7
0
```

— 範例一說明 —

第一次詢問時 $B_1 = 0$ 。接著操作 3 讓 B_1 增加 -7 ，所以第二次詢問得到 -7 。最後操作 5 撤銷這次修改，使 B_1 恢復為 0 。

— 範例二輸入 —

4 11
1 5 5 10
3 5 10 4
1 1 2
4 1
2 2 3
4 3
5
4 3
5
4 1
5
4 2

— 範例二輸出 —

4
8
4
0
0

— 範例二說明 —

第一次操作後， $B = (0, 4, 4, 4)$ 。合併點 1, 2 所在的連通塊後，詢問點 1 得到 4。接著把點 2 移到點 3 所在的連通塊，該連通塊的總和為 $4 + 4 = 8$ 。

第一次撤銷會還原移動操作，所以詢問點 3 得到 4；第二次撤銷會還原合併操作，所以詢問點 1 得到 0；第三次撤銷會還原最初的區間修改，所以詢問點 2 也得到 0。

— 範例三輸入 —

```
3 13
2 4 6
1 1 2
1 1 2
3 3 3 5
3 2 6 0
4 1
5
3 2 2 7
2 1 2
4 2
5
4 2
5
4 1
```

— 範例三輸出 —

```
0
7
0
0
```

— 範例三說明 —

第一次合併點 1, 2 的操作成功，但緊接著再次合併同一對點不會成功。後面的兩次區間修改也分別因為區間內沒有符合的點與 $x = 0$ 而不會成功；詢問操作同樣不會進入回溯紀錄。因此第一次撤銷會直接撤銷最初那次成功的合併。

接著對 $A_i = 2$ 的點增加 7，再把點 1 移入點 2 所在的連通塊，詢問得到 7。撤銷移動後，點 2 自己的 B 值為 0；再撤銷區間修改後，點 1 的 B 值也恢復為 0。

G. Performance Analysis

Problem ID: Compiler

Time Limit: 1s

Memory Limit: 512MiB

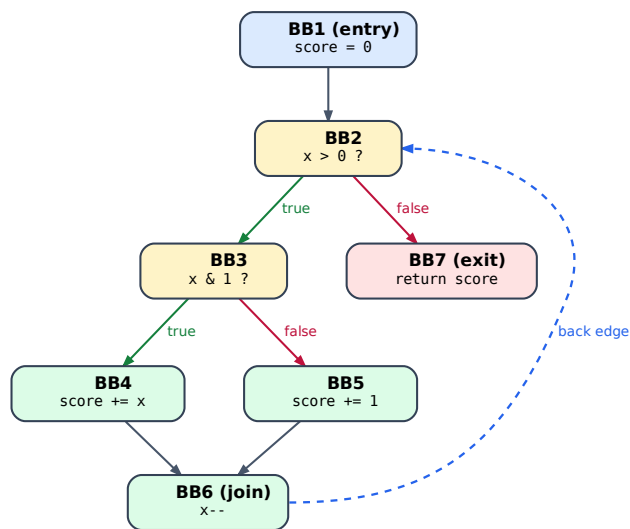


Figure 1: 一個含迴圈與條件分支的 CFG。每個框是一個 basic block； BB_6 是兩條分支的 join point， $BB_6 \rightarrow BB_2$ 是迴圈的 back edge。

tobiichi3227 跟 yushiuan9499 最近在開發 NCC，tobiichi3227 發現它程式執行太慢了，所以想給 NCC 加上一個功能，統計每一條 control-flow edge 在程式執行期間總共被通過幾次。

編譯器將程式用控制流圖 (Control Flow Graph，後面簡稱 CFG) 表示，CFG 上面會有 N 個基本塊 (Basic Block，後面簡稱 BB)，每個 BB 會有幾個有向邊連到其他 BB，代表從 BB_i 可以走到 BB_j ，總共有 M 條有向邊。

程式每執行一次，控制流都會從入口 1 開始，沿著 CFG 上的有向邊前進，最後抵達出口 N 。過程中每沿著一條有向邊走過一次，該邊的通過次數就增加 1；若因為迴圈而在同一次執行中重複通過同一條邊，則每次通過都會分別計數。程式總共執行 R 次，令 f_e 表示這 R 次執行合計通過邊 e 的次數。

對於每條邊，編譯器在它所連接的 BB 上插入性能計數器 (BB_i 或 BB_j 都可以，編譯器只會選一個 BB 插入)，因為測量通過次數會造成額外的運算，所以插入每個計數器需要成本 c_e ，執行結束後就能知道該邊通過的次數。

每次控制流進入一個普通 BB 後，都一定會再沿著某條邊離開。因此，普通 BB 被進入的總次數會等於它被離開的總次數。入口 1 是每次執行的起點，所以離開次數會比進入次數多 R ；出口 N 則正好相反。也就是說：

- 對於入口與出口以外的每個 BB，所有連進該 BB 的邊，其通過次數總和，等於所有從該 BB 連出去的邊，其通過次數總和。

- 對於入口 1 ，所有連出去的邊的通過次數總和，等於所有連進來的邊的通過次數總和加上 R 。
- 對於出口 N ，所有連進來的邊的通過次數總和，等於所有連出去的邊的通過次數總和加上 R 。

利用這些進出次數的關係，可以從已知的邊通過次數推導出部分沒有安裝計數器的邊。

求最小安裝成本，使得執行結束後能夠**唯一確定每一條邊通過的次數**。

— 輸入 —

第一行會有三個整數 N, M, R ，代表 BB 的數量、邊的數量與執行次數

接下來 M 行會有三個整數 u, v, C ，代表 BB_u 可以走到 BB_v ，且安裝計數器成本為 C

— 輸出 —

最小安裝成本

— 輸入限制 —

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 2 \times 10^5$
- $2M \leq R \leq 10^{18}$
- $1 \leq u, v \leq N$
- $1 \leq C_e \leq 10^9$
- 允許 self-loop 與重邊
- 保證所有 BB 一定有一個 1 到 N 的路徑通過

— 子任務 —

編號	分數	額外限制
1	0	範例輸入輸出
2	5	$M = N - 1$, 且第 i 條邊為 $i \rightarrow i + 1$
3	10	$C_e = 1$
4	15	$M \leq 20$
5	15	$M = N$ 且無 self-loop 或重複連接
6	20	每個點雙連通分量皆為單邊分量，或其中每個頂點的分量內度數皆為 2
7	35	無額外限制

— 範例輸入 —

4 4 8
1 2 5
1 3 2
2 4 4
3 4 6

— 範例輸出 —

2

— 範例解釋 —

輸入的 CFG 的結構如下：

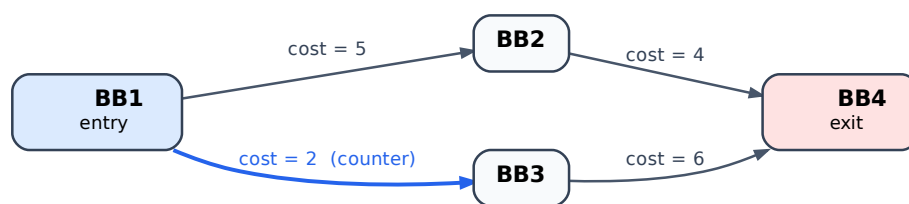


Figure 2: 範例一的 CFG。邊上標示安裝計數器的成本；藍色粗邊表示在 $1 \rightarrow 3$ 安裝計數器。

程式總共執行 8 次。每次執行會選擇下列其中一條路線：

$1 \rightarrow 2 \rightarrow 4$ 或 $1 \rightarrow 3 \rightarrow 4$

可以花費 2，在邊 $1 \rightarrow 3$ 上安裝計數器。

假設程式執行完畢後，計數器顯示邊 $1 \rightarrow 3$ 總共被通過 3 次。因為程式總共執行 8 次，所以邊 $1 \rightarrow 2$ 必定被通過 $8 - 3 = 5$ 次。

每次進入 BB_2 後都會沿著邊 $2 \rightarrow 4$ 離開，因此邊 $2 \rightarrow 4$ 也被通過 5 次。

同理，邊 $3 \rightarrow 4$ 被通過 3 次。

因此，只要在邊 $1 \rightarrow 3$ 上安裝一個計數器，就能確定所有邊的通過次數。

若不安裝任何計數器，則無法判斷 8 次執行中分別有多少次經過 BB_2 或 BB_3 。因此至少需要安裝一個計數器。

所有邊中最低的安裝成本為 2，所以最小總費用為 2。

— 範例輸入 —

4 7 14
1 2 8
2 4 5
1 3 7

3 4 4

2 3 6

2 2 3

1 2 9

— 範例輸出 —

21

— 範例輸入 —

6 6 12

1 2 9

2 3 8

3 4 3

2 4 7

4 5 6

5 6 5

— 範例輸出 —

3

— 範例輸入 —

8 9 18

1 2 10

2 3 8

3 5 2

2 4 6

4 5 5

5 6 7

6 7 4

7 8 3

6 8 9

— 範例輸出 —

5

－ 編譯器小教室：從 C Code 到 CFG －

C 的 if、while、for 等語法適合人類閱讀，但編譯器進行最佳化時，更希望控制流只有少數幾種明確操作：

```
goto L
if condition goto L_true else goto L_false
return value
```

把結構化控制流展開成 label 與 jump 的過程，通常稱為 **lowering**。下文使用的 goto-style 是為了教學設計的 pseudo IR：它和 LLVM IR 一樣用 label 與明確的 branch 描述控制流，但它**不是合法的 LLVM IR**。真正的 LLVM IR 有型別、SSA、phi 等規則，而且使用的 terminator 名稱是 br、ret、switch 等。

概念上的對應關係如下：

```
goto L
    <=> br label %L

if c goto T else goto F
    <=> br i1 %c, label %T, label %F

return x
    <=> ret i32 %x
```

把大括號中的 if、迴圈等結構展開成 label 與 goto，概念上正是 TOJ 756 練習的工作。即使考場沒有網路，也不影響閱讀以下內容；連結只供課後延伸。

Basic Block (BB)

Basic block 是一段**最大且連續**的指令序列，具有以下性質：

- 控制流只會從第一條指令進入，不會跳進 block 中間；
- 除了最後一條指令以外，中途不會跳到別處；
- block 內的指令一旦開始，就會照順序全部執行到 terminator。

在 LLVM-like IR 中，每個 BB 以 label 開始，以一條 terminator 結束。terminator 決定下一個 BB，或直接結束函式。

將一串 goto-style 指令切成 BB 時，可以先找出 **leaders**：

1. 函式的第一條指令是 leader；
2. 任何 jump 的目標 label 是 leader；
3. conditional branch、goto、return 後面的下一條指令若存在，也是新 block 的 leader。

每個 leader 到下一個 leader 之前的所有指令，會形成一個 BB。實作時通常會再刪除 unreachable blocks，並合併不必要的空 blocks。

從 BB 建立 CFG

CFG 中每個節點是一個 BB。若執行完 BB_u 後可能接著執行 BB_v ，就在圖上加入有向邊 $u \rightarrow v$ ：

- goto BB2：一條邊連到 BB_2 ；
- if c goto BB2 else BB3：分別連到 BB_2 與 BB_3 ；
- return：沒有函式內的 successor；
- 若 IR 允許 fall-through，則連到文字順序的下一個 BB。本文會把 jump 全部寫出來，避免隱藏的 fall-through。

因此，goto-style 的 label 先決定 BB 邊界，terminator 再決定 CFG edges。

常見 C 控制結構如何 Lower ?

以下例子都先改寫成 goto-style，再從每個 terminator 直接讀出 CFG。

1. 直線程式

```
x = a + b;
y = x * 2;
return y;
```

沒有 branch，整段就是一個 BB：

BB1:

```
    x = a + b
    y = x * 2
    return y
```

return 是 terminator，因此不能再把其他可執行指令接在它後面。

2. if / else

```
if (x < 0)
    y = -x;
else
    y = x;
use(y);
```

Lowering 後：

```
BB1: if x < 0 goto BB2 else goto BB3
BB2: y = -x; goto BB4
BB3: y = x;  goto BB4
BB4: use(y); ...
```

CFG edges 為 $1 \rightarrow 2$ 、 $1 \rightarrow 3$ 、 $2 \rightarrow 4$ 、 $3 \rightarrow 4$ 。BB₄ 是兩條分支重新會合的 **join point**。若原始程式沒有 else，則 BB₁ 的 false edge 可以直接連到 join point。

3. while

```
while (x > 0) {  
    sum += x;  
    --x;  
}  
use(sum);
```

Lowering 後：

```
BB1:  
    goto BB2  
  
BB2:  
    if x > 0 goto BB3 else goto BB4  
  
BB3:  
    sum += x  
    --x  
    goto BB2  
  
BB4:  
    use(sum)  
    ...
```

BB_2 是 loop header，每次迭代前都檢查條件。 $3 \rightarrow 2$ 讓 CFG 形成 cycle； $2 \rightarrow 4$ 則是離開迴圈的 exit edge。

4. `do / while`

```
do {  
    read_next();  
} while (!ok);  
use_data();
```

Lowering 後：

BB1:

```
    read_next()  
    goto BB2
```

BB2:

```
    if !ok goto BB1 else goto BB3
```

BB3:

```
    use_data()  
    ...
```

條件放在 `body` 後方，所以 BB_1 至少會執行一次；這就是它和 `while` CFG 的主要差異。

5. for、break 與 continue

```
for (int i = 0; i < n; ++i) {  
    if (a[i] < 0) continue;  
    if (a[i] == 0) break;  
    sum += a[i];  
}  
use(sum);
```

for (init; cond; step) body 可以拆成 init、condition、body、step 與 exit：

```
BB1: i = 0; goto BB2  
BB2: if i < n      goto BB3 else goto BB7  
BB3: if a[i] < 0   goto BB6 else goto BB4  
BB4: if a[i] == 0  goto BB7 else goto BB5  
BB5: sum += a[i]; goto BB6  
BB6: ++i; goto BB2  
BB7: use(sum); ...
```

continue 必須跳到 step 的 BB_6 ，不能直接跳回 condition，否則會漏掉 ++i。break 則直接跳到 loop exit BB_7 。

6. Short-circuit : && 與 ||

```
if (a != 0 && b / a > 2)
    hit();
next();
```

&& 只有在左側為 true 時才會計算右側，所以兩個判斷通常位於不同 BB：

BB1:

```
if a != 0 goto BB2 else goto BB4
```

BB2:

```
if b / a > 2 goto BB3 else goto BB4
```

BB3:

```
hit()
goto BB4
```

BB4:

```
next()
...
```

這張 CFG 也解釋了為什麼 $a == 0$ 時不會計算 b / a 。對 $a || b$ 而言則相反：左側為 true 時直接走 true edge，只有左側為 false 才計算右側。

7. switch

```
switch (op) {  
case 0: y = 10; break;  
case 1: y = 20; break;  
default: y = -1;  
}  
use(y);
```

最單純的 lowering 是一串比較：

```
BB1: if op == 0 goto BB3 else goto BB2  
BB2: if op == 1 goto BB4 else goto BB5  
BB3: y = 10; goto BB6  
BB4: y = 20; goto BB6  
BB5: y = -1; goto BB6  
BB6: use(y); ...
```

真正的編譯器也可能保留多路 switch terminator，或產生 jump table。無論機器碼如何實作，CFG 的 dispatch block 都有多個可能 successors，各個 case 最後再連到 join point。

8. 提早 return 每個 return 都會立刻結束目前 BB，而且在函式內沒有 successor。若某個分析希望只有一個出口，可以先將回傳值存入暫存變數，再全部跳到共用出口：

```
BB_return_negative:  
    result = -1  
    goto BB_exit  
  
BB_return_answer:  
    result = answer  
    goto BB_exit  
  
BB_exit:  
    return result
```

這是 compiler pass 常見的 CFG normalization，但不是所有 compiler 都必須這樣做。

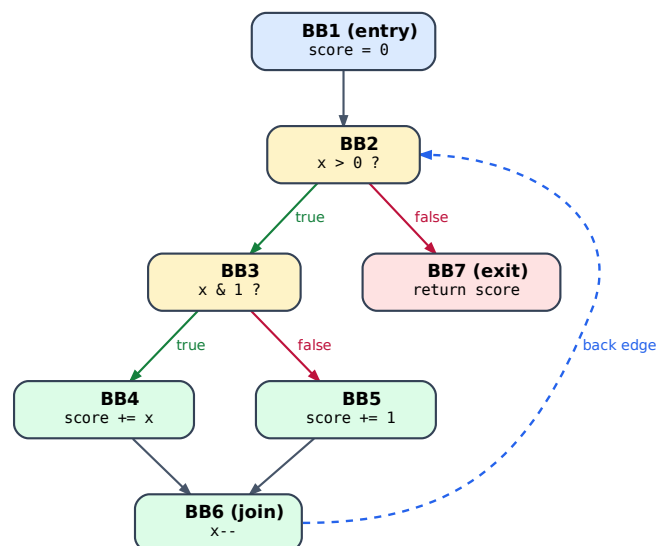
完整範例：從 C、goto-style 到 CFG

題目開頭的 CFG 對應以下 C function：

```
int calc(int x) {
    int score = 0;
    while (x > 0) {
        if (x & 1)
            score += x;
        else
            score += 1;
        --x;
    }
    return score;
}
```

先 lower 成只含明確 jump 的 pseudo IR：

```
BB1: score = 0; goto BB2
BB2: if x > 0 goto BB3 else goto BB7
BB3: if x & 1 goto BB4 else goto BB5
BB4: score += x; goto BB6
BB5: score += 1; goto BB6
BB6: --x; goto BB2
BB7: return score
```



上述 goto-style pseudo IR 建立出的 CFG。

BB_4 與 BB_5 不能合併，因為兩者是不同 branch targets。 BB_6 也必須從 join point 開始，因為它同時有 BB_4 、 BB_5 兩個 predecessors。最後， $BB_6 \rightarrow BB_2$ 回到 loop header，形成迴圈。

支配關係 (Dominance)

在具有入口的 CFG 中，若從 entry 到 v 的**每一條路徑**都一定經過 u ，就稱 u **支配** v ，記為 $u \text{ dom } v$ 。

- 每個節點都支配自己；
- 若 $u \neq v$ 且 u 支配 v ，稱為 strict dominance；
- v 的 **immediate dominator**，記為 $\text{idom}(v)$ ，是最靠近 v 的 strict dominator。

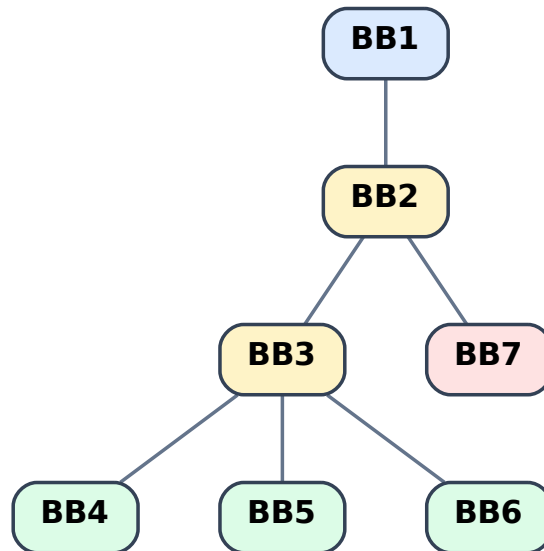
以前面的七個 BB 為例：

節點	Dominators	idom
BB_1	$\{BB_1\}$	無
BB_2	$\{BB_1, BB_2\}$	BB_1
BB_3	$\{BB_1, BB_2, BB_3\}$	BB_2
BB_4	$\{BB_1, BB_2, BB_3, BB_4\}$	BB_3
BB_5	$\{BB_1, BB_2, BB_3, BB_5\}$	BB_3
BB_6	$\{BB_1, BB_2, BB_3, BB_6\}$	BB_3
BB_7	$\{BB_1, BB_2, BB_7\}$	BB_2

BB_4 不支配 BB_6 ，因為可以沿著 $BB_3 \rightarrow BB_5 \rightarrow BB_6$ 到達 BB_6 而不經過 BB_4 。同理， BB_5 也不支配 BB_6 ；但不論走哪一臂都一定先經過 BB_3 ，所以 $\text{idom}(BB_6) = BB_3$ 。

Dominator Tree

把每個非 entry 節點 v 接到 $\text{idom}(v)$ ，就得到 dominator tree：



完整範例的 dominator tree。父節點是子節點的 immediate dominator。

在 dominator tree 中， u 支配 v ，恰好等價於 u 是 v 的 ancestor。注意 dominator-tree edges 代表的是「必經關係」，不一定是原 CFG 中的 control-flow edges。

Dominance 是許多 compiler analyses 的基礎，例如：

- 確認 SSA value 的 definition 是否支配它的 uses；
- 配合 dominance frontier 決定需要放置 phi 的 join points；
- 判斷某段運算能否安全移動；
- 找出 loop header 與 natural loop。

Dominance 與 Loop

對 CFG edge $u \rightarrow h$ ，若 h 支配 u ，則這條邊稱為 **back edge**， h 通常是 loop header。完整範例中， BB_2 支配 BB_6 ，所以 $BB_6 \rightarrow BB_2$ 是 back edge。

這條 back edge 對應的 natural loop 為：

{ BB_2 , BB_3 , BB_4 , BB_5 , BB_6 }

BB_2 是 header， BB_6 是跳回 header 的 latch， $BB_2 \rightarrow BB_7$ 是離開迴圈的 exit edge。
 BB_7 不在 loop 內。

一般有向圖中的 cycle 不一定都具有單一、支配整個 cycle 的 header；這類 CFG 稱為 irreducible。由 while、for 等結構化語法產生的迴圈，通常會形成較容易分析的 reducible CFG。

回到本題

本題已經直接給出 CFG，不需要自行把 C code lowering 成 goto-style。輸入的每個點就是一個 BB，每條有向邊就是一種可能的控制轉移；一次程式執行，就是從 entry 1 沿 CFG 走到 exit N 。性能計數器所測量的，正是這些 control-flow edges 實際被通過的次數。

115 學年度資訊學科能力競賽臺南一中校內複選
**115 學年度資訊學科能力競賽臺南一中校內複選 試
題本**

附錄

祝各位都能破台，GL & HF